

What must be remembered about a deleted character?

v2: positions are testimony; only ties force retention
(working note — throwaway)

Sal / FP Launchpad — KC + Claude

2026-07-14

The question. When a character is deleted from a replicated list, what must the system remember about it, and for how long? The requirement throughout (“the contract”): once any user has seen character u displayed before character v , no user may ever see v before u ; and replicas with the same events must agree.

Correction from v1. Version 1 argued that retention of a dead timestamp is forced whenever a deleted character has living descendants. That is true for designs whose merge *re-renders positions from identity keys*, but KC’s proposal — keep every node’s absolute range untouched through deletes and merges — refutes the general claim: on v1’s own example it produces the right answer with *nothing* retained. This version records the corrected boundary, machine-checked: **distinct positions are permanent testimony; retention is forced only at ties.**

Encoding. Every node stores its full absolute range $Q = (lo, hi) \subseteq (0, 1)$. Insert takes the lower half of the remaining gap inside the parent’s absolute range (first child of the root: $(0, \frac{1}{2})$). Display: depth-first, children by descending lo , equal lo broken by newer timestamp first. Characters are named by their timestamps (larger = later). Under KC’s proposal, delete removes the record and re-parents the children; *no number ever changes*.

1 The four-event example — and why it forces nothing

step	action	screen
1	type d (time 1)	d
2	type a (time 4), same position — <i>both replicas sync</i> —	$a\ d$ $a\ d$
3	Alice: type x (time 6) <i>after</i> d	$a\ d\ x \Rightarrow$ Alice has seen a before x
4	Bob: delete d	a
5	<i>merge</i> M_1 (LCA = the synced version)	?

Ranges: $d = (0, \frac{16}{32})$, $a = (\frac{16}{32}, \frac{24}{32})$, $x = (0, \frac{8}{32})$ (lower half of d). Four designs at M_1 :

design	what survives d ’s death	M_1	verdict
RGA	d kept as a tombstone	$a\ x$	✓
always keep the dead time	stamp (1) on x	$a\ x$	✓
keep only if d conflicted	nothing (d never conflicted)	$x\ a$	✗ flips Alice’s (a, x)
keep the ranges (KC)	nothing — x stays at $(0, \frac{8}{32})$	$a\ x$	✓

The keep-range row is the correction: x ’s position $(0, \frac{8}{32})$, below a ’s $(\frac{16}{32}, \frac{24}{32})$, is d ’s testimony — carried geometrically, permanently, for free. The conditional design fails here not because information must be retained but because it re-renders from keys and then forgets the key. This example refutes conditional forgetting in re-render designs; it does *not* force retention on position-keeping designs.

2 Keep-the-range survives everything the conditional designs failed

Machine-checked (regressions from `litmus/contest_tree.py` run against the keep-range design):

countermodel	killed	keep-range
4-event example (above)	conditional retention	survives: $a\ x$
subordination escape (7 events)	contest-only stamps	survives: (4, 6) order preserved
retroactive subordination (6 events)	condition-on-current-level stamps	survives: (3, 2) order preserved

The reason is uniform: in all three histories every deciding comparison is between *distinct* ranges, and kept ranges never change — so every verdict ever displayed is re-derivable from live positions alone, at every replica, forever.

3 The one failing shape: ties

Concurrent characters typed at the same position from identical states mint *identical* ranges — position carries zero information between them. Machine trace (the L25 shape, three branches):

step	state
R_0 : type 6 at root	$6 = (0, \frac{16}{32})$
R_M (concurrent): type 10 at root	$10 = (0, \frac{16}{32}) \leftarrow \text{identical mint: a tie}$
$M_1 = \text{merge}$: tie broken by time	display $[10, 6]$
type 16, 22 under 6	$16 = (0, \frac{8}{32})$, $22 = (\frac{8}{32}, \frac{12}{32})$, both $\subset (0, \frac{16}{32})$ display $[10, 6, 22, 16]$ (10 before 22 and 16)
R_D (from R_0): delete 6	
final merge, ranges kept	display $[22, 16, 10]$ both pairs flipped

The mechanism: 6’s range *equals* 10’s, so “inside 6” is numerically also “inside 10” — 22’s kept range has $lo = \frac{8}{32} > 0 = 10$ ’s lo , and the position sort puts 22 *above* 10. The verdict “10 before 6” was a fact about the *pair of timestamps*, decided at M_1 ; the heirs’ sub-ranges nest inside *both* rivals’ equal ranges and cannot stay subordinate by geometry once their side of the tie dies. At random-DAG scale keep-range fails 52/120 executions — this shape is the conjectured root of all of them (minimization pending).

4 The sibling-chain connection

The Q-ranges and the sibling links are two encodings of one structure, and the tie analysis says exactly where they coincide and where they part.

sibling-link view	Q-range view
node’s birth pointers: (parent, displaced sibling)	node’s mint range
distinct chain positions	distinct ranges
two nodes displacing the same head	identical mint ranges — a tie
splicing a dead node out of the chain	deleting a record, ranges kept
recording the spliced link through a dead node	the tie stamp / dig

The carve is deterministic: same parent and same displaced head $\Rightarrow$ the same slice. So *tie-ness in ranges is exactly equality of the birth pointers* — a tie is two nodes claiming the same chain position, and the range encoding is the sibling chain *arithmetized*. Both encodings order distinct chain positions for free and both run out of positional information at the same place: a shared chain position, where only timestamps can decide. And the retention question is the same question in both wardrobes: keeping the tie verdict for a dead node’s heirs *is* keeping the spliced-out link through the dead node.

The two encodings genuinely differ in one behavior (machine-checked, `litmus/sibling_tree.py`): what happens to the *displacers* of a tied node. Branch *A* stacks 2 then 5 over 1; branch *B* concurrently stacks 4 over 1 — so 2 and 4 tie (both displaced 1), and 5 displaced 2. The range read sorts the level globally: [5, 4, 2, 1] (5 is newest, floats to the top). The sibling read keeps 5 glued above the 2 it displaced: [4, 5, 2, 1]. Sequentially the encodings agree on 300/300 random histories; they part exactly at ties. Run-gluing is the Fugue-direction behavior — a different datatype, not a different encoding of this one.

5 The sharpened claim

Retention is forced only at ties. Distinct concurrent mints are ordered by permanent geometry — equivalently, by their positions in the sibling chains. Identical concurrent mints (shared chain positions) are ordered by a timestamp verdict that some state must carry for as long as either side has living descendants — as a tombstone (RGA), a stamp (always-keep), a retained spliced link (sibling chains), or any equivalent.

This re-opens a door the conditional-retention refutations appeared to close. Their principle was: *the decision whether heirs inherit a dead rank must be a function of immutable data* — “was it contested” and “was it subordinated” are mutable, concurrently accruing knowledge, and both conditions fell to countermodels. But **tie-ness is immutable**: mint ranges (equivalently, birth pointers) are never rewritten, so “*u* and *v* claimed the same position” is a fixed fact about the pair. A design that keeps ranges verbatim and stamps heirs *only when their dead ancestor was tied* is therefore a live candidate, not a refuted one. Its retention would be $O(\text{tie families})$: zero on a purely sequential million-character history, growing only with genuine same-position races.

Pre-registered risks (the machine decides; not yet run). (i) *Posthumous ties*: a later insert can coincidentally mint a dead node’s exact range (machine-observed) — though such pairs were never co-displayed, so their order may be legitimately free. (ii) *Tied orphans*: a tie whose dead side was born and deleted inside one branch, invisible to the merge that meets the survivor. (iii) *Mixed-comparator transitivity*: position where distinct, stamps at ties — the combined relation must remain a total order. (iv) *Nested tie families*: heirs of both sides of a tie meeting each other.

6 Where the LCA helps — scoped honestly

The MRDT merge sees three states; “dead in a branch, live in the LCA” is visible for free, timestamp included — power that two-way-merge CRDTs lack. v1 argued this covers only the first merge after a death, with retention forced one merge later. That argument stands *only for re-render designs*: keep-the-range needs no consultation at all for distinct-range deaths, and for tie deaths the merge that processes the death typically still holds the tied record in an input (in the trace above, the final merge sees 6 live in one input). The honest residue is exactly risk (ii): ties whose dead side no input remembers.

Status. Machine-checked: remembering nothing at all breaks — at ties (8 refuted designs); conditional retention on mutable conditions breaks (2 refuted rules); always-remembering works and equals the tombstoned RGA ($O(\text{history})$ worst case); keep-the-range works everywhere except ties (52/120 at scale). Open: tie-only retention (the candidate above), and stability-gated discard of whatever is retained.

Provenance. `whiteboard/litmus/`: `contest_tree.py` (designs and countermodel regressions, and the always-keep control’s lockstep equivalence with the tombstoned RGA), `sibling_tree.py` (the encoding-divergence probe). The keep-range runs reproduce from the session transcript of 2026-07-14; all table values machine-printed.